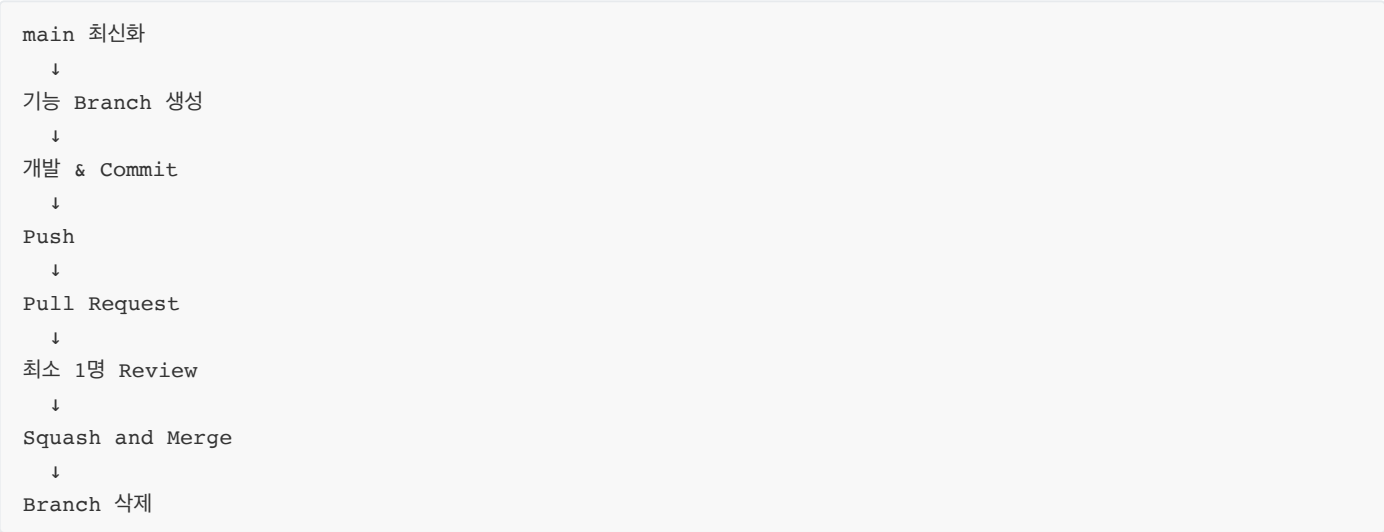


Git Convention

1. 작업 흐름



- `main` 에서 직접 개발하거나 Push하지 않습니다.
- 모든 작업은 기능별 Branch에서 진행합니다.
- 작업 완료 후 Pull Request(PR)를 통해 `main` 에 반영합니다.

2. Branch 규칙

형식

type/기능명

Type

Type	용도
<code>feat</code>	새로운 기능
<code>fix</code>	버그 수정
<code>refactor</code>	기능 변경 없는 코드 개선
<code>chore</code>	Docker, 설정, 빌드 등
<code>docs</code>	문서
<code>test</code>	테스트

예시

```
feat/login
feat/user-profile
feat/company-search
feat/company-analysis

fix/login-error
fix/company-response

chore/docker
docs/api-spec
```

- 기능명은 짧은 영문 소문자와 - 를 사용합니다.
- develop, release 등의 별도 장기 Branch는 사용하지 않습니다.

작업 시작

```
git checkout main
git pull origin main
git checkout -b feat/login
```

3. Commit 규칙

형식

type(scope): 내용

Type

Type	용도
feat	새로운 기능
fix	버그 수정
refactor	리팩토링
docs	문서
test	테스트
chore	환경설정, 빌드, Docker 등
ci	CI 설정

Scope

Scope	의미
fe	Frontend
be	Backend
db	Database
ai	AI
api	API
infra	Docker 및 개발환경
docs	문서

예시

```
feat(fe): 로그인 화면 구현
feat(be): 회원가입 API 구현
feat(db): 사용자 테이블 추가
feat(ai): 기업 추천 분석 기능 구현

fix(be): 중복 이메일 검증 오류 수정
chore(infra): Docker Compose 구성
docs(api): 기업 조회 API 명세 작성
```

원칙

- **Commit 하나 = 하나의 작업 목적**
- 무엇을 변경했는지 알 수 있게 작성합니다.
- 수정, 작업, 최종 처럼 의미가 불분명한 메시지는 사용하지 않습니다.
- Commit 전에 변경 파일을 확인합니다.

```
git status
git add .
git status
git commit -m "feat(fe): 로그인 화면 구현"
git push -u origin feat/login
```

4. Pull Request / Review / Merge

PR 제목

Commit과 동일한 형식을 사용합니다.

```
feat(be): 기업 목록 조회 API 구현
```

PR 내용

작업 내용

- 기업 목록 조회 API 구현

테스트

- 정상 응답 확인

참고사항

- 리뷰어가 알아야 할 내용이 있으면 작성

Frontend 화면 변경이 있으면 필요한 경우 Screenshot을 첨부합니다.

Review / Merge

- `main` Merge 전 최소 1명의 Review를 받습니다.
- Merge 방식은 **Squash and Merge**를 사용합니다.
- Merge 완료 후 작업 Branch는 삭제합니다.

5. 주의사항

- `main` 직접 Push 금지
- Force Push 금지
- 동작하지 않는 코드 Merge 금지
- `.env`, API Key, 비밀번호 등 Secret Commit 금지
- 다른 팀원의 변경사항을 이유 없이 삭제하거나 되돌리지 않기

Quick Reference

Branch

`feat/login`

Commit

`feat(fe): 로그인 화면 구현`

PR

`feat(fe): 로그인 화면 구현`

Merge

Squash and Merge

Rule

PR + 최소 1명 Review 후 main Merge